

MFCC

COURSE OUTLINE

- Introduction to MFCC (Mel-Frequency Cepstral Coefficients)
- Create MFCC using Audio Library

INTRODUCTION

- MFCC (Mel Frequency Cepstral Coefficients) adalah metode untuk mengekstrak feature-feature dari audio.
 - Fitur: properti atau karakteristik terukur dari suatu data, digunakan sebagai input untuk train model dan membuat prediksi.
 - Dalam konteks speech recognition, fitur adalah representasi numerik dari karakteristik utama sinyal suara yang bisa digunakan oleh model untuk membedakan atau mengenali suara.
- MFCC menggunakan MEL scale untuk membagi pita frequency menjadi pita-pita yang lebih kecil, lalu mengekstrak Cepstral Coefficients.

MEL SCALE?

EKSPERIMEN SEDERHANA

- Tone Generator: <https://www.szynalski.com/tone-generator/>
- Dengarkan perbedaan pitch antara:
 - C2 (65Hz) ke C4 (262Hz) ... [Selisih 197Hz]
 - G6 (1568Hz) ke A6 (1760Hz) ... [Selisih 192Hz]

MEL SCALE?

EKSPERIMEN SEDERHANA

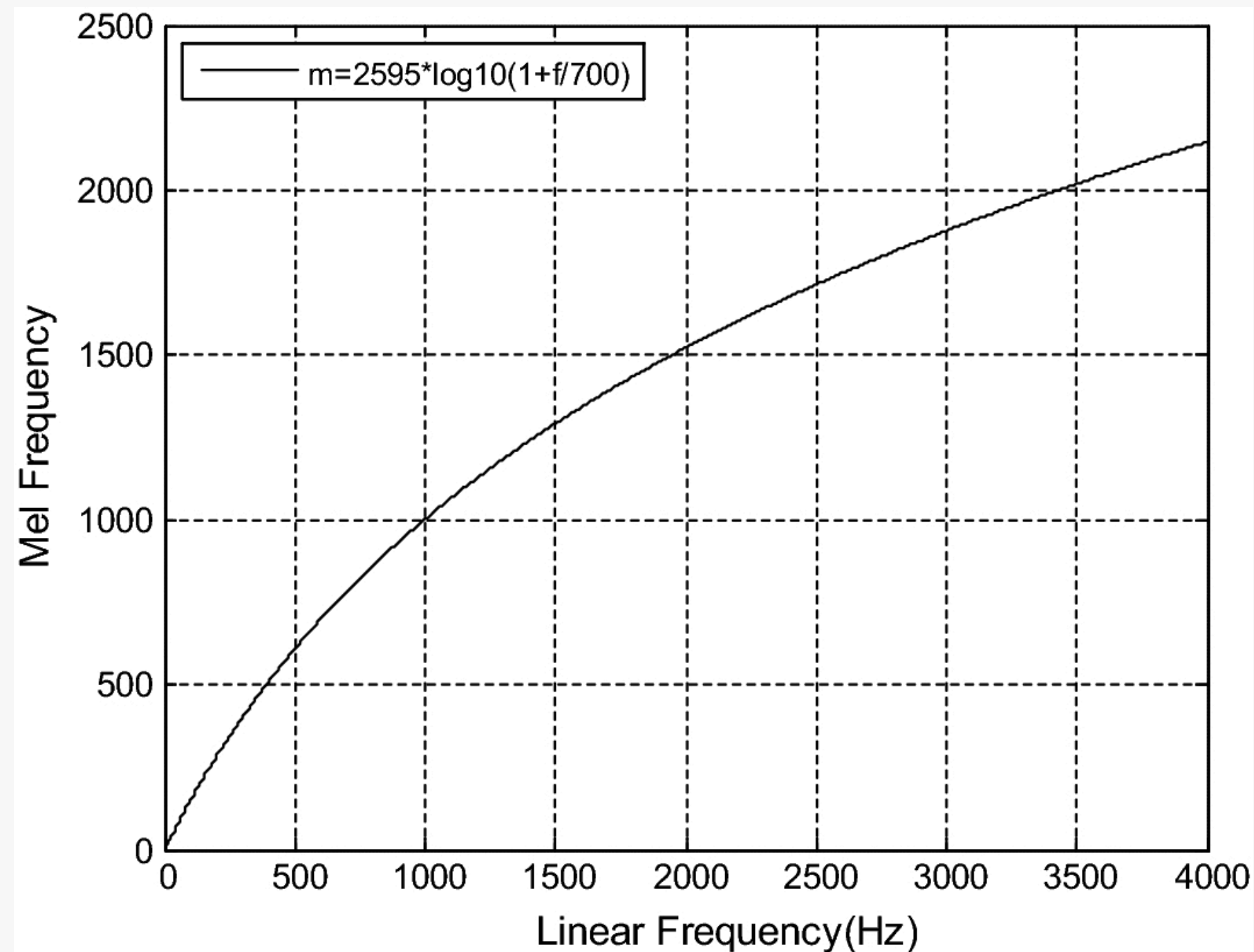
- Perbedaan antara C2 (65Hz) ke C4 (262Hz) terdengar lebih drastis dari G6 (1568Hz) ke A6 (1760Hz) meskipun jarak frekuensinya kira-kira sama (+/- 200Hz).
- Hal ini dikarenakan persepsi manusia akan suara berbeda-beda di setiap frekuensi.

MEL SCALE?

- Perhatikan kalimat di bawah ini:
 - Perbedaan nada 100Hz ke 300Hz berbeda dengan nada 1100Hz ke 1300Hz, di mana perbedaan dari 100Hz ke 300Hz akan terdengar lebih drastis.
 - Akan tetapi, perbedaan nada 100Mel ke 300Mel akan terdengar sama dengan 1100Mel ke 1300Mel.

MEL SCALE?

- Mel Scale adalah skala frekuensi yang dirancang untuk meniru persepsi manusia terhadap suara.
- Skala ini logaritmik di frekuensi tinggi dan hampir linear di frekuensi rendah.
- 1000 Mel = 1000 Hz. Titik ini dijadikan titik referensi dalam konversi antara Hz dan Mel.



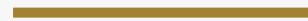
CEPSTRAL COEFFICIENTS

- Adalah kumpulan fitur-fitur yang umum digunakan untuk speech processing dan recognition. (Hasil akhir proses ekstraksi fitur pada suara)
- Dikalkulasikan menggunakan cepstrum.

LANGKAH-LANGKAH MFCC

- Normalization: menstandarisasi amplitudo sinyal suara agar berada dalam rentang tertentu (suara lebih seragam).
- Frame Blocking: memecah suara menjadi potongan-potongan kecil (frame).
- Windowing: menghindari distorsi karena pemotongan frame (memperhalus transisi antar frame).
- Fast Fourier Transform (FFT) : mengubah sinyal dari time domain ke frequency domain (mengubah suara ke bentuk frekuensi).
- Mel Filterbank: mengubah spektrum frekuensi ke mel scale.
- Discrete Cosine Transform (DCT): mengambil fitur utama untuk analisis.

CODE



IMPORT LIBRARY

```
# Import Library
import numpy as np # untuk kalkulasi matematika
from scipy.io import wavfile # untuk input audio dari file wav
import scipy.fftpack as fft # untuk Fast Fourier Transform (FFT)
from scipy.signal import get_window # untuk windowing
import matplotlib.pyplot as plt # untuk plotting hasil MFCC
```

READ AUDIO FILE

```
# Read Audio File
TRAIN_PATH = './sample_audio.wav'
sample_rate, audio = wavfile.read(TRAIN_PATH)
audio_duration = len(audio) / sample_rate
print(f'Sample rate: {sample_rate}')
print(f'Audio duration: {audio_duration}')
```

NORMALIZATION

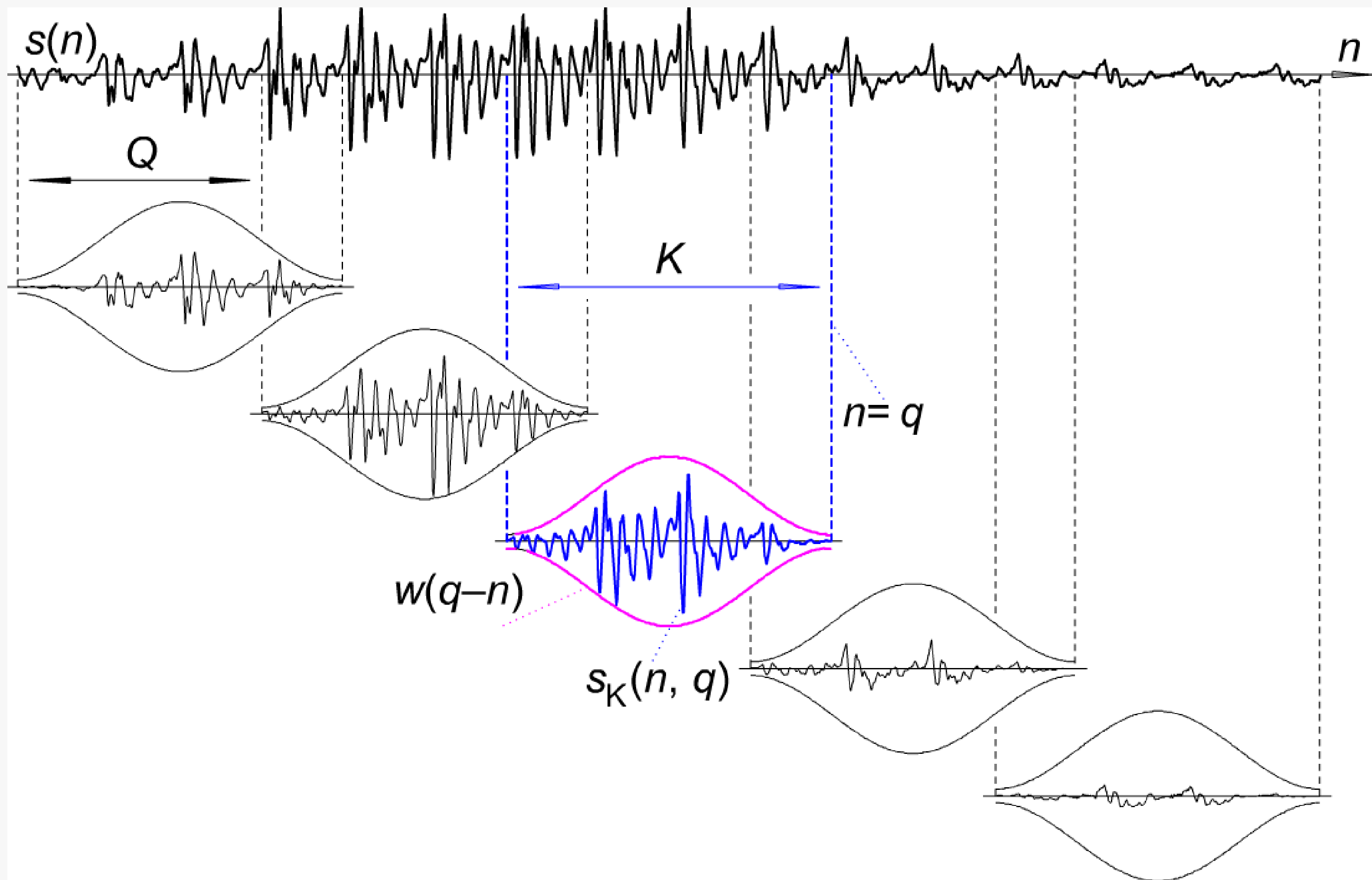
- Kita ingin mengatur level dari audionya agar amplitudonya dari -1 sampai 1.
- Kita melakukan normalisasi agar sinyal memiliki volume yang konsisten di seluruh rekaman.
- Untuk itu, kita akan membagi audio dengan nilai maximum absolute dari audio.

```
# Normalization
```

```
# membagi audio dengan nilai maximum absolute dari audio  
audio = audio / np.max(np.abs(audio))
```

PLOTTING

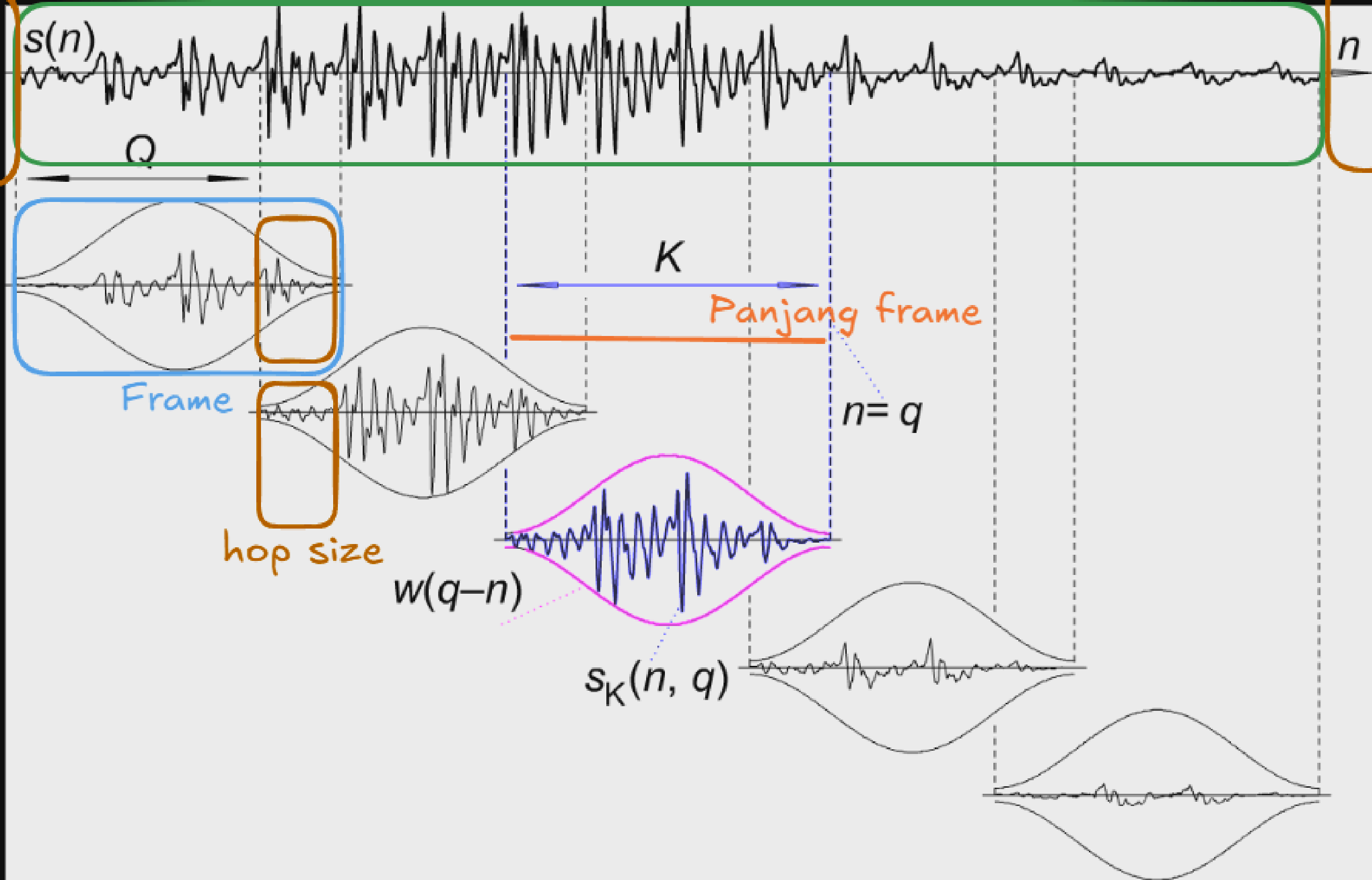
```
# Plotting audio waveform  
plt.figure(figsize=(15,4))  
plt.plot(np.linspace(0, audio_duration, num=len(audio)), audio)  
plt.grid(True)
```



Padding

Panjang Audio

Padding

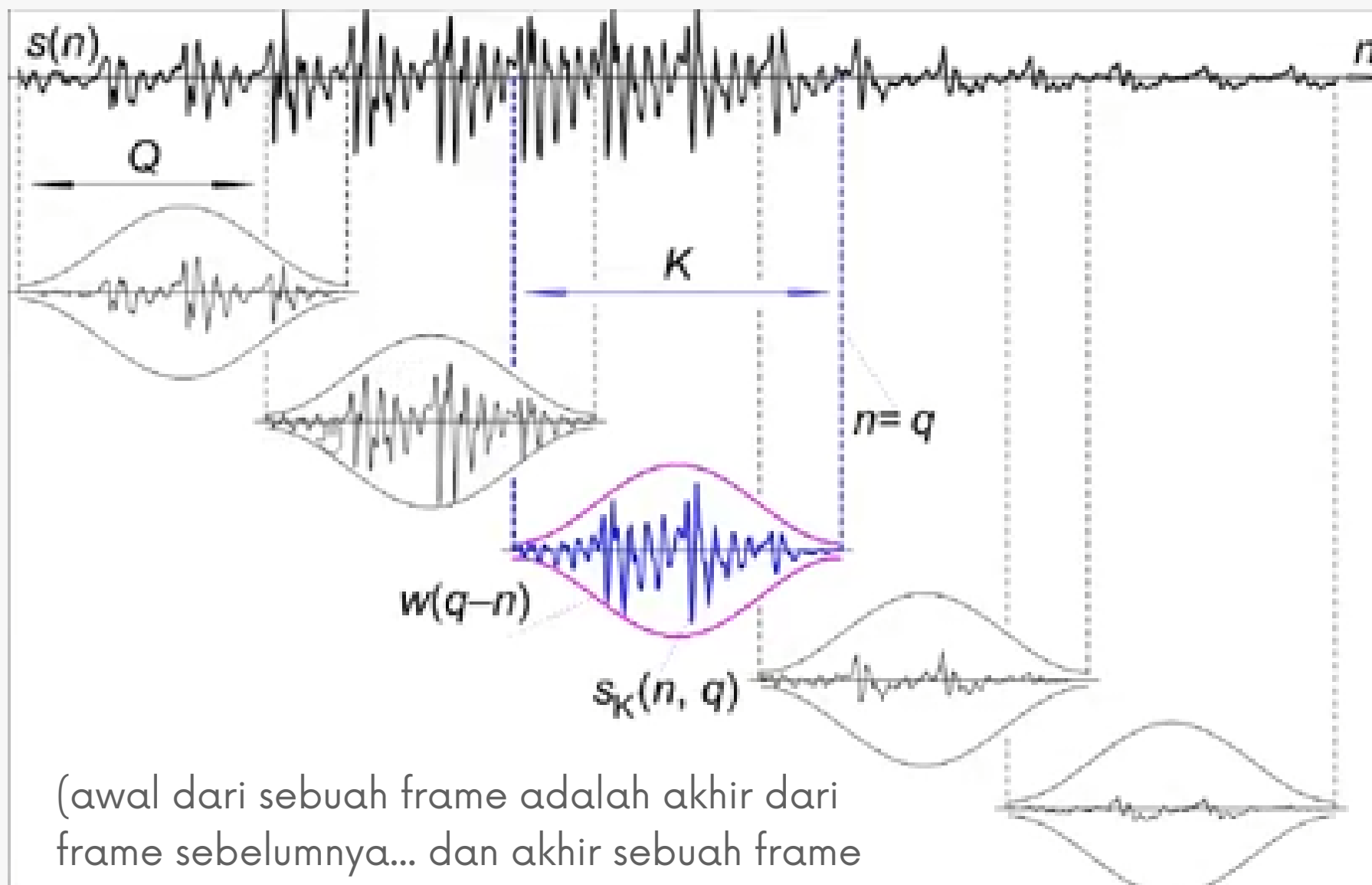


FRAME BLOCKING

- Penerapan FFT digunakan untuk mengubah sinyal dari domain waktu ke domain frekuensi. Namun, suara merupakan sinyal **non-stasioner** (karakteristik sinyal berubah seiring waktu).
- Jika FFT diterapkan langsung pada keseluruhan sinyal suara, hasilnya dapat mengandung distorsi (perubahan yang tidak diinginkan dalam sinyal asli) karena FFT bekerja dengan asumsi bahwa sinyal yang dianalisis bersifat **stasioner** (karakteristik sinyal tidak berubah terhadap waktu).

FRAME BLOCKING

- Jadi, sinyal audio dibagi menjadi segmen-segmen kecil (frame) dalam durasi yang sangat pendek (misalnya 20-40 ms).
- Dalam durasi yang sangat pendek, sinyal suara dapat dianggap stasioner, sehingga FFT dapat bekerja secara lebih akurat dalam setiap frame.
- Frame saling tumpang tindih (overlapping) untuk menjaga kontinuitas informasi dan memastikan adanya korelasi antar frame, sehingga transisi antar bagian suara tetap mulus.



(awal dari sebuah frame adalah akhir dari frame sebelumnya... dan akhir sebuah frame adalah awal frame setelahnya)

FRAME BLOCKING

```
# Frame Blocking

def frame_audio(audio, FFT_size, hop_size, sample_rate):
    audio = np.pad(audio, int(FFT_size/2), mode='reflect')

    # panjang setiap audio frame dalam jumlah sample:
    frame_len = np.round(sample_rate * (hop_size / 1000)).astype(int)

    # jumlah frame yang dihasilkan:
    frame_num = int((len(audio) - FFT_size) / frame_len) + 1

    # kerangka frame (matrix kosong untuk menyimpan hasil frame blocking):
    frames = np.zeros((frame_num, FFT_size))

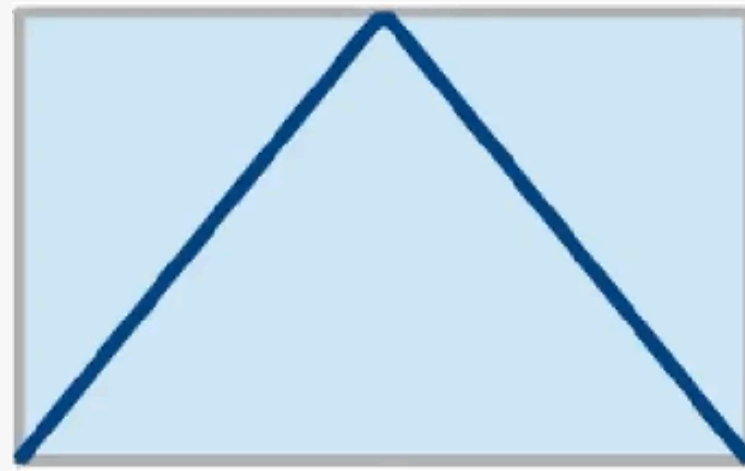
    # mengisi frame dengan masing-masing bagian dari audio sepanjang FFT_SIZE:
    for i in range(frame_num):
        frames[i] = audio[i * frame_len : i * frame_len + FFT_size]

    return frames
```

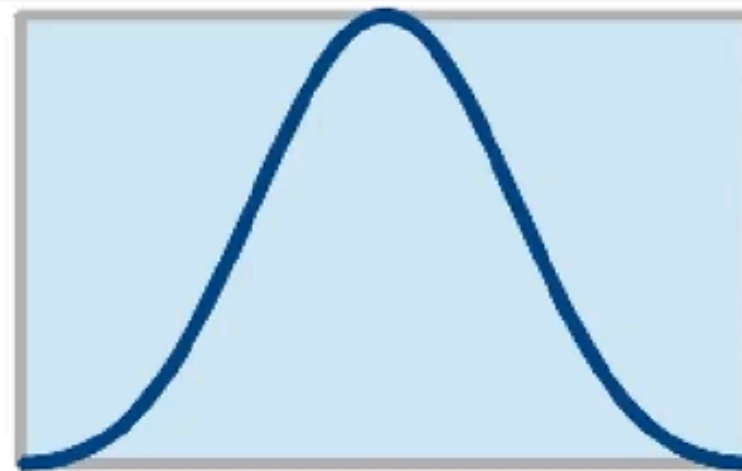
WINDOWING

- Sekarang, audio kita berada di time domain (terhadap waktu). Kita ingin mengubahnya ke frequency domain (terhadap frekuensi). Untuk itu, kita harus menggunakan FFT.
- Untuk melakukan FFT, audio yang diberikan harus periodic (sudah dari periodic setelah frame blocking) dan continuous. Untuk membuat audio menjadi continuous, kita menggunakan window function pada setiap frame.
- Window function adalah function yang membuat ujung awal dan akhir dari sinyal kita dekat dengan 0, sehingga tidak ada lompatan nilai yang menimbulkan distorsi.

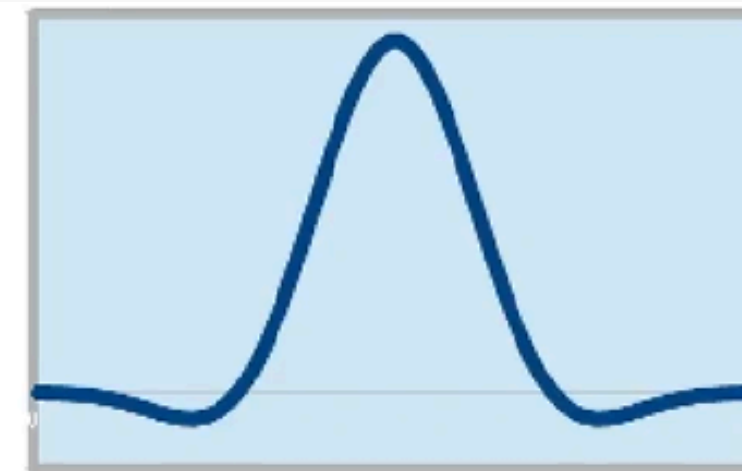
WINDOWING



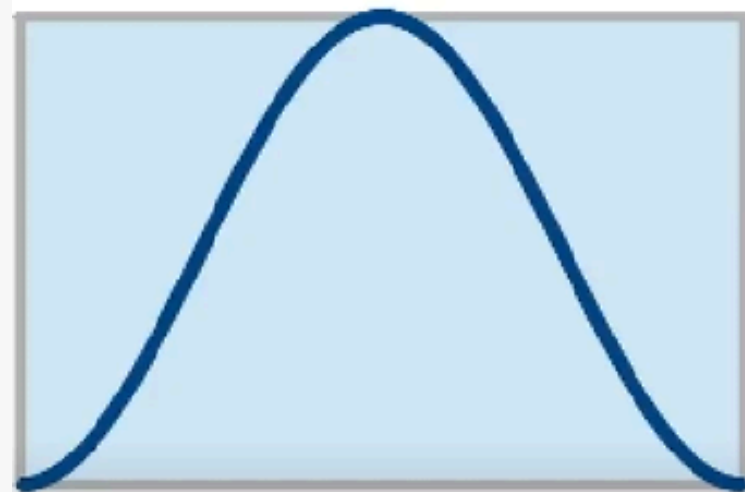
Bartlett



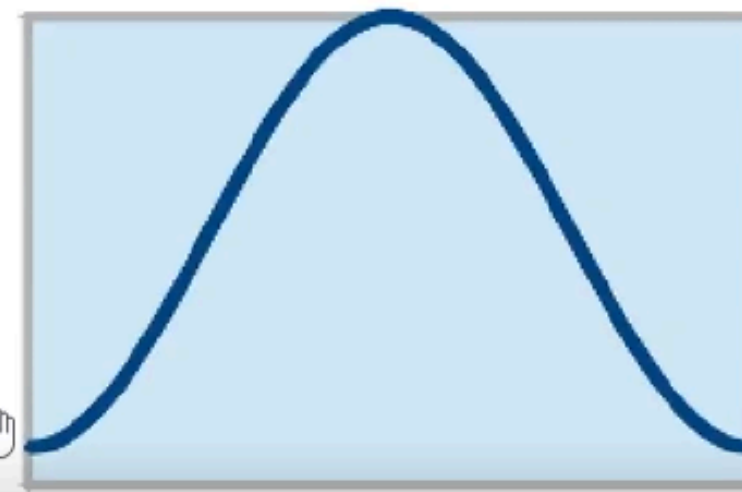
Blackman



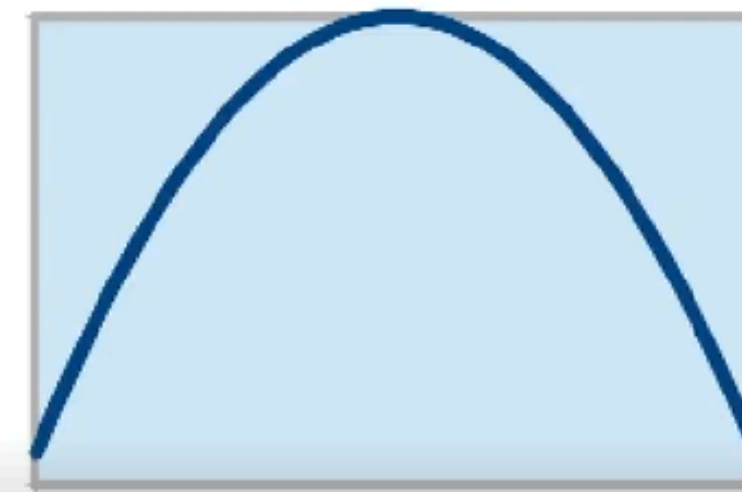
Flat top



Hanning



Hamming



Welch

WINDOWING

```
# Windowing

# window jenis hanning
window = get_window("hann", FFT_size, fftbins=True)

# apply window function ke frame yang kita buat
audio_win = audio_framed * window
```

FAST FOURIER TRANSFORM

- Mengubah audio dari time domain ke frequency domain.

```
# FFT

# transpose audio (baris: frekuensi, kolom: frame)
audio_winT = np.transpose(audio_win)
print(audio_winT.shape)

# array kosong untuk audio yang sudah di FFT
audio_fft = np.empty((int(1 + FFT_size // 2), audio_winT.shape[1]), dtype=np.complex64, order='F')

# algoritma FFT:
for i in range(audio_fft.shape[1]): # iterasi terhadap column (total frame)
    audio_fft[:, i] = fft.fft(audio_winT[:, i], axis=0)[: audio_fft.shape[0]]

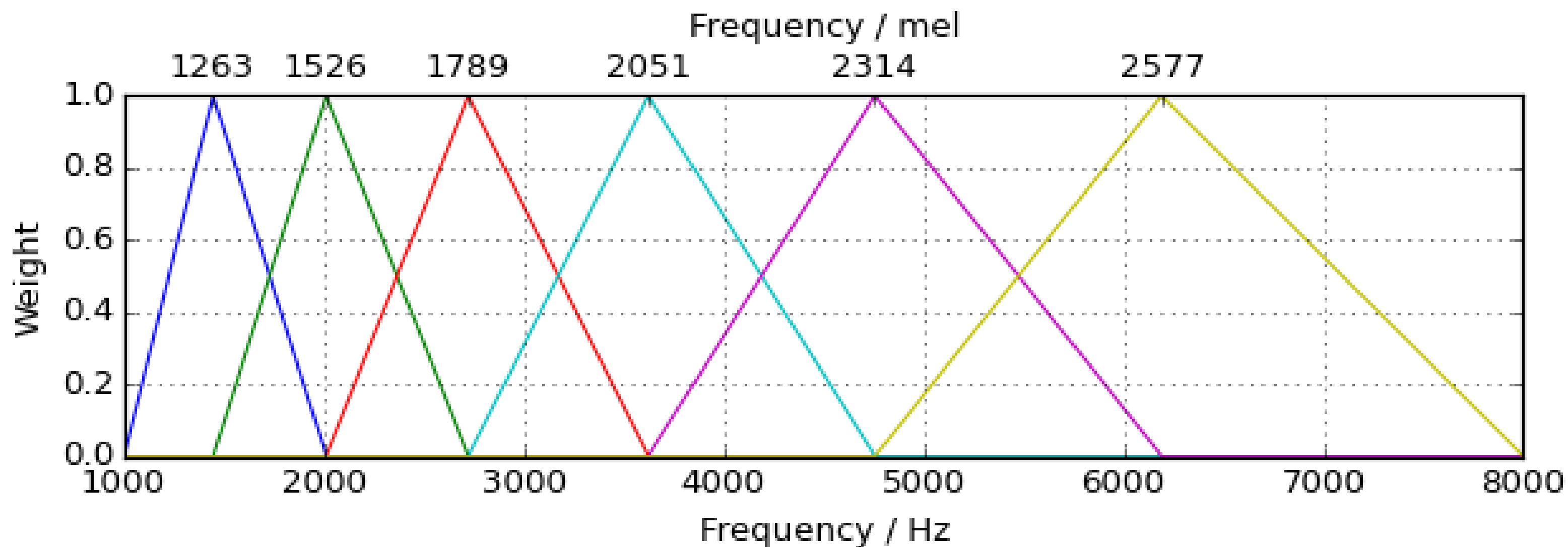
# transpose kembali audio_fft (baris: frame, kolom: frekuensi)
audio_fft = np.transpose(audio_fft)
print(audio_fft.shape)

# kalkulasi signal power
audio_power = np.square(np.abs(audio_fft))
print(audio_power.shape)
```

MEL FILTERBANK

- Mel Filterbank adalah kumpulan filter yang diaplikasikan ke power spectrum dari sebuah sinyal audio. Setiap filterbank merepresentasikan energi dari sinyal di suatu frekuensi pada skala mel.
- Tujuan dari Mel Filterbank adalah untuk mengubah power spectrum dari audio signal yang awalnya ada pada frequency domain (FFT) menjadi format Mel Frequency.

MEL FILTERBANK



MEL FILTERBANK

```
# Mel Filterbank  
  
freq_min = 0  
freq_high = sample_rate / 2 # Nyquist-Shannon sampling theorem  
mel_filter_num = 10 # bebas
```

MEL FILTERBANK

- Untuk mencegah aliasing, sampling rate harus minimal $2\times$ dari frequency tertinggi di sebuah sinyal.
- Aliasing: sebuah sinyal dengan frekuensi lebih tinggi dari separuh frekuensi sampling tidak dapat direpresentasikan dengan benar, sehingga muncul sebagai distorsi atau sinyal baru dengan frekuensi yang lebih rendah.
- Aliasing terjadi ketika sampling rate terlalu rendah sehingga sinyal tinggi muncul sebagai sinyal lebih rendah yang salah.

MEL FILTERBANK

```
# Freq to Mel
def freq_to_mel(freq):
    return 2595.0 * np.log10(1.0 + freq / 700.0)

# Mel to Freq
def mel_to_freq(mels):
    return 700.0 * (10 ** (mels / 2595.0) - 1)
```

MEL FILTERBANK

```
# Function untuk mencari titik-titik dari filter
# (nantinya untuk membuat filter)
def get_filter_points(fmin, fmax, mel_filter_num, FFT_size, sample_rate):
    fmin_mel = freq_to_mel(fmin)
    fmax_mel = freq_to_mel(fmax)

    # Membuat barisan dari fmin_mel ke fmax_mel
    mels = np.linspace(fmin_mel, fmax_mel, num=mel_filter_num + 2)

    # Convert mels balik ke freq
    freqs = mel_to_freq(mels)

    # Return titik-titik filternya dengan rumus:
    return ((FFT_size + 1) / sample_rate * freqs).astype(int), freqs
```

```
filter_points, mel_freqs = get_filter_points(freq_min, freq_high, mel_filter_num, FFT_size, sample_rate)
print(filter_points)
print(mel_freqs)
```

MEL FILTERBANK

```
# Membuat filter

def get_filters(filter_points, FFT_size):
    # exclude fmin_mel dan fmax_mel
    # menyimpan nilai yang positif saja, karena hasil FFT simetris
    filters = np.zeros((len(filter_points) - 2, int(FFT_size / 2 + 1)))

    for n in range(len(filter_points) - 2):
        # nilai saat naik: 0 ke 1 increasing order
        # mengisi filters dengan dimensi 0 nya adalah n, dan dimensi 1 nya adalah filter_
points ke n sampai n + 1 (nilai setelahnya)
        # dengan angka-angka antara 0 - 1 sebanyak filter_points ke n+1 dikurang filter_p
oints ke n
        filters[n, filter_points[n] : filter_points[n+1]] = np.linspace(0, 1, filter_poin
ts[n+1] - filter_points[n])

        # nilai saat turun: 1 ke 0 decreasing order
        filters[n, filter_points[n+1] : filter_points[n+2]] = np.linspace(1, 0, filter_po
ints[n+2] - filter_points[n+1])

    return filters
```

```
filters = get_filters(filter_points, FFT_size)
```

MEL FILTERBANK

```
# Normalisasi: menghindari noise seiring naiknya freq
# Menyesuaikan amplitudo dari setiap filter
# 2 / (freq titik awal - freq titik akhir)
enorm = 2.0 / (mel_freqs[2 : mel_filter_num + 2] - mel_freqs[:mel_filter_num])

# Apply normalisasi
print(filters.shape)
print(enorm.shape)

enorm = enorm[:, np.newaxis]
print(enorm.shape)

filters *= enorm
```

MEL FILTERBANK

```
# Convert power spectrum dari audio signal
audio_filtered = np.dot(filters, np.transpose(audio_power))
audio_log = np.log10(audio_filtered) * 10.0 # mengubah ke satuan decible (dB)
print(audio_log.shape)
```

DCT

- Langkah terakhir untuk mencari MFCC, untuk mendapatkan Cepstral Coefficients.
- Proses matematis yang akan mengubah energi dari filterbank yang sudah dihitung menjadi serangkaian koefisien.
- Menggunakan DCT-III, mengekstrak perubahan frekuensi tinggi dan frekuensi rendah di sinyalnya.

DCT

```
# DCT

def dct(dct_filter_num, filter_len):
    basis = np.empty((dct_filter_num, filter_len))

    # Inisialisasi baris pertama basis dengan nilai 1 dibagi sqrt(filter_len)
    basis[0, :] = 1.0 / np.sqrt(filter_len)

    samples = np.arange(1, 2 * filter_len, 2) * np.pi / (2.0 * filter_len)

    for i in range(1, dct_filter_num):
        basis[i, :] = np.cos(i * samples) * np.sqrt(2.0 / filter_len)

    return basis
```

DCT

```
dct_filter_num = 40  
dct_filters = dct(dct_filter_num, mel_filter_num)
```

```
cepstral_coefficients = np.dot(dct_filters, audio_log)  
cepstral_coefficients.shape
```

DCT

```
# Show image dari Cepstral Coefficient  
# dapat direpresentasikan secara visual  
plt.figure(figsize=(15,4))  
plt.imshow(cepstral_coefficients, aspect='auto', origin='lower')
```

THANK YOU

